Ctrl+Z Burnout

Academic Decision Simulator

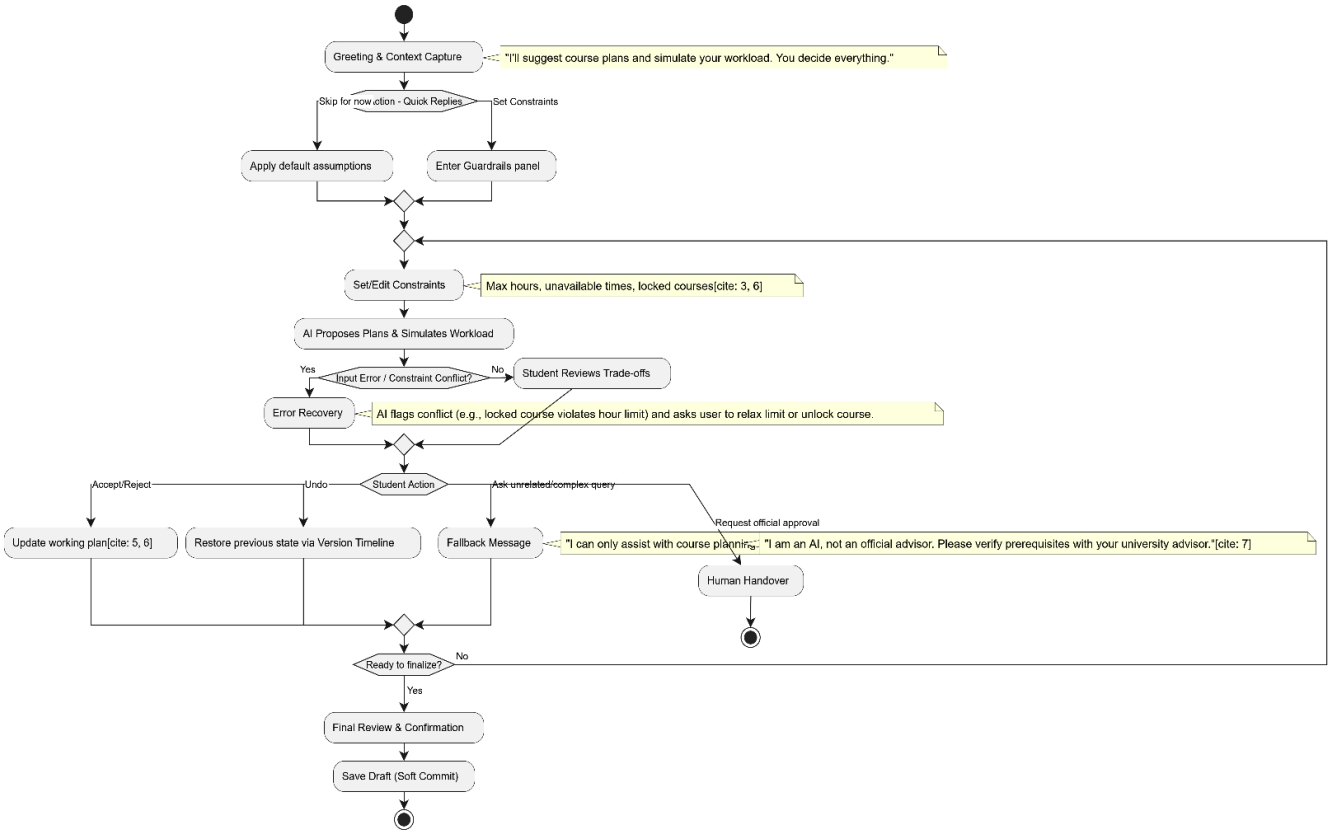
Week 5: Flow and Architecture

- **Team:** CTRL+Z BurnOut
- **Members:**

1. Aya Ahmed Hussien
2. Toka Alaa Ebraheem
3. Toka Nasr Saeed
4. Mohammed Mostafa Mohammed
5. Youssef Hesham Abdelfattah

1. Conversation Flow

1.1 Flow Diagram



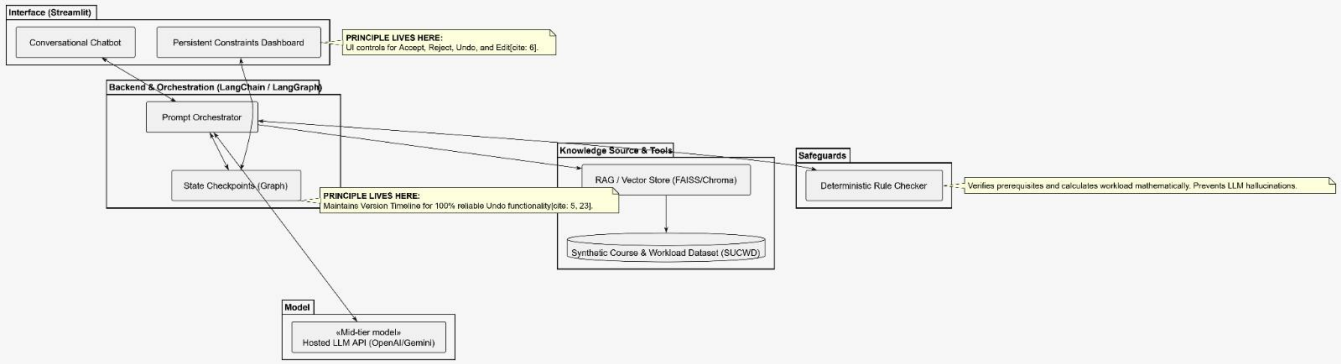
1.2 Conversation Flow

- The conversation starts with a greeting and context capture, where the chatbot explains its purpose and allows the student to either set constraints or skip this step using quick replies.
- The system then applies default assumptions or allows the student to enter the Guardrails Panel to set or edit constraints, such as maximum study hours, unavailable times, and locked courses.
- After setting the constraints, the AI proposes course plans and simulates the workload. If an input error or constraint conflict occurs, the system provides error recovery and asks the student to adjust the conflicting constraints.

- The student then reviews the proposed plans and trade-offs and can:
  - **Accept/Reject:** Update the working plan.
  - **Undo:** Restore the previous state using the Version Timeline.
  - **Ask unrelated or complex queries:** Receive a fallback message, or be directed to human handover when official approval is required.
- Finally, the student decides whether the plan is ready to be finalized. If not, the process returns for further adjustments. If yes, the student proceeds to Final Review & Confirmation, and the plan is saved as a draft (Soft Commit).

2. System Architecture

2.1 Architecture Diagram



2.2 Components

The system architecture consists of five main components:

1. **Interface (Streamlit):** Includes a Conversational Chatbot and a Persistent Constraints Dashboard, which provides controls for Accept, Reject, Undo, and Edit.
2. **Backend & Orchestration (LangChain / LangGraph):** Includes a Prompt Orchestrator and State Checkpoints to manage the system's workflow and maintain a version timeline.
3. **Model:** Uses a hosted LLM API (OpenAI/Gemini) to process user inputs and generate responses.
4. **Knowledge Source & Tools:** Includes a RAG / Vector Store (FAISS/Chroma) connected to the Synthetic Course & Workload Dataset (SUCWD).
5. **Safeguards:** Includes a Deterministic Rule Checker that verifies prerequisites and performs mathematical workload calculations to prevent LLM hallucinations.

2.3 Principle Feature

The Persistent Constraints Management feature is located in the Interface through the Persistent Constraints Dashboard, supported by State Checkpoints in the Backend, which maintain a version timeline and enable Undo functionality.

3. Shneiderman’s 8 Golden Rules Checklist

Each rule is matched to the screen or flow decision that meets it.

#	Rule	Screen / flow decision that meets it	Met?
1	**Strive for consistency**	Every change follows the same loop (Set/Edit Constraints, AI proposes, student reviews). Accept, Reject, Undo and Edit look and behave the same in the Persistent Constraints Dashboard. Conflict messages in Error Recovery always name the conflicting constraint and the two ways out: relax the limit or unlock the course.	**Yes**
2	**Enable shortcuts**	"Skip for now" quick reply applies default assumptions and goes straight to a first plan. One-tap Accept or Reject per course. The Guardrails panel can be reopened at any time to edit limits.	**Yes**
3	**Offer informative feedback**	Every accepted change or edited constraint triggers an updated workload simulation. Error Recovery explains conflicts in plain words. The Version Timeline shows which version of the plan the student is looking at.	**Yes**
4	**Design dialogs that give closure**	Final Review & Confirmation, then Save Draft (Soft Commit), ends the main path with a clear confirmation. Human Handover ends with a message telling the student to verify with their university advisor.	**Yes**
5	**Prevent errors and handle them simply**	The Guardrails panel sets limits before any suggestion appears. The Deterministic Rule Checker verifies prerequisites and workload math, so the LLM cannot invent a prerequisite. Error Recovery handles constraint conflicts, and the Fallback Message keeps the bot within course planning.	**Yes**
6	**Permit easy reversal of actions**	Undo restores the previous state from the Version Timeline (State Checkpoints in the backend). Reject keeps the current plan and works per course (granular reject). Saving is a Soft Commit, so nothing is final.	**Yes**
7	**Support the user's sense of control**	The greeting says "You decide everything". The student chooses to set constraints or skip, accepts or rejects each course, edits limits at any time, and can ask for official approval through Human Handover. The AI never relaxes a stated constraint silently.	**Yes**
8	**Reduce short-term memory load**	Quick replies replace typed commands. Constraints stay visible in the Persistent Constraints Dashboard. The Version Timeline lets the student return to earlier plans instead of remembering them. Trade-offs are shown with each proposed plan.	**Yes**

4. First Model Response from Our Own Backend

Here’s a live demo that shows how it responds: First Model Response (video)

Appendix A: Prompt Log

#	Tool	Exact prompt	What it gave	What we changed and why
1	**Claude**	Act as a Lead AI System Architect and UX Researcher. I am continuing the development of the "Academic Decision Simulator" (Team: Ctrl+Z Burnout). Our core HAI Principle is "User Control and Agency." Our tech stack includes Streamlit (UI), LangChain/LangGraph (Orchestration), an LLM API, and a RAG pipeline querying our synthetic SUCWD dataset. Please generate the deliverables for "Week 5: Flow and Architecture" based on the following strict requirements: 1. Conversation Flow Diagram: Provide a detailed textual outline (and a Mermaid.js flowchart code block) showing the user journey. It MUST explicitly include: • The Greeting. • Main Paths (Inputting constraints -> Generating 3 drafts -> Reviewing). • Quick Replies (e.g., Constraint chips, "Undo", "Reject"). • Error Recovery (e.g., LLM hallucinates a prerequisite). • The Fallback Message (e.g., system doesn't understand a complex rule). • Human Handover (e.g., routing to a human Academic Advisor for credit overloads). 2. System Architecture Diagram: Provide a textual outline (and a Mermaid.js architecture diagram code block) detailing the system components. It MUST show: • Interface (Streamlit). • Backend (LangChain/LangGraph orchestration). • Model (LLM API). • Knowledge Source (Vector DB with SUCWD). • Safeguards (Deterministic Rule Checker for prerequisites/credits). • CRITICAL: Explicitly mark/highlight exactly where our principle feature ("User Control and Agency" / Undo state management) lives within this architecture (e.g., in LangGraph state checkpoints and Streamlit session state). 3. Shneiderman's 8 Golden Rules Checklist: Create a table mapping each of the 8 rules to our specific UI/flow decisions. Make sure the rules "Permit easy reversal of actions" and "Support user's sense of control" heavily reference our Undo button, Granular Reject, and Edit Limits features. 4. First Model Response (Backend Demo Script): Write a minimal, clean Python script	Generated detailed textual outlines for the Conversation Flow and System Architecture, Mermaid.js code blocks for diagram generation, a mapped checklist for Shneiderman's 8 Golden Rules, and a Python (Streamlit + LangChain) script for the backend demo.	**Change:** Adjusted the conversation flow diagram to clearly emphasize the "Guardrails panel" and the "Error Recovery" loop. Re-mapped Shneiderman's rules (specifically rules 6 and 7) to explicitly reference our custom "Version Timeline" and "Undo" features. **Why:** To align perfectly with our core HAI principle (User Control) and accurately reflect the custom architecture we designed.

#	Tool	Exact prompt	What it gave	What we changed and why
		using streamlit and langchain (with ChatOpenAI or similar) that creates a simple UI. The user types a greeting, and the LLM responds as the "Academic Decision Simulator". This code must be copy-pasteable so my team can run it locally (streamlit run app.py) and record the live backend call for the professor.		